

Graph Deep Learning for Time Series Processing

McGill University

Andrea Cini

March 10, 2025

Visiting SNSF Postdoc @ University of Oxford



**Swiss National
Science Foundation**

What is this about?



Traffic monitoring



Smart cities



Energy analytics



Physics



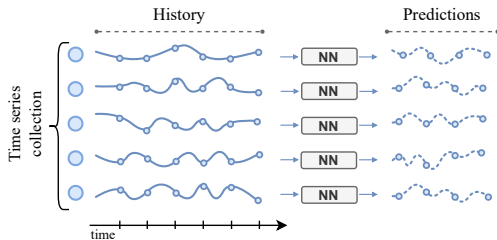
Stock markets

What is this about?

Deep learning for time series forecasting

Modern deep learning forecasting methods rely on a **single neural network** trained on a collection of **related time series**.

- 😊 Each time series is processed **independently**.
- 😊 Parameters are **shared**.
- 😊 Effective and **sample efficient**.
- 😞 **Dependencies are neglected**.



[1] Salinas *et al.*, “DeepAR: Probabilistic forecasting with autoregressive recurrent networks”, IJF 2020.

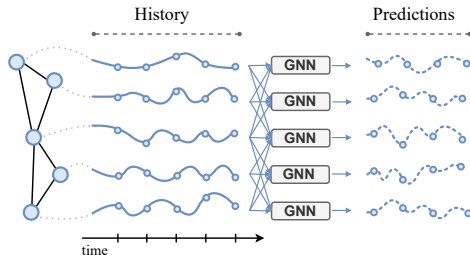
[2] Benidis *et al.*, “Deep Learning for Time Series Forecasting: Tutorial and Literature Survey”, ACM CSUR 2022.

What is this about?

Graph deep learning for time series forecasting

Graph deep learning (GDL) provides appropriate operators to go beyond these limitations.

- 😊 **Dependencies** are **embedded into the processing** as inductive biases.
- 😊 Operate on **sets of correlated time series**.
- 😊 Parameters are **shared**.



- 😞 There are inherent **challenges** in applying this processing to data from the real world.

[3] Cini et al., “Graph Deep Learning for Time Series Forecasting”, preprint 2023.

What is this about?

Presentation outline

Part 1

- 1.1)** Correlated time series
- 1.2)** Forecasting models
- 1.3)** Graph-based representation
- 1.4)** STGNN architectures

Part 2

- 2.1)** Local effects
- 2.2)** Dealing with missing data
- 2.3)** Scalability



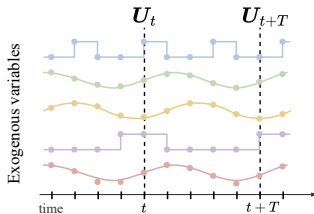
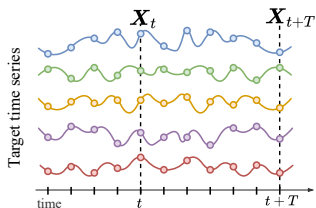
Conclusions

Forecasting correlated time series

Collections of time series

Consider a set $\mathcal{D}$ of N **correlated time series**. Each i -th time series can be associated with:

- **observations** $x_t^i \in \mathbb{R}^{d_x}$ at each time step t ;
- **exogenous variables** $u_t^i \in \mathbb{R}^{d_u}$ at each time step t ;
- a vector of **static (time-independent) attributes** $v^i \in \mathbb{R}^{d_v}$.



Static attributes



Capital letters denote the stacked N time series, i.e., $X_t \in \mathbb{R}^{N \times d_x}$, $U_t \in \mathbb{R}^{N \times d_u}$.

→ We call **spatial** the dimension spanning the collection.

Correlated time series

We can consider a **time-invariant** stochastic process generating each time series as

$$\mathbf{x}_t^i \sim p^i(\mathbf{x}_t^i | \mathbf{X}_{<t}, \mathbf{U}_{\leq t}, \mathbf{V}) \quad \text{for all } i = 1 \dots N, t = 0, \dots, T-1$$

and assume the existence of a **causality à la Granger** among time series.

Furthermore time series

- are assumed
 - a) homogenous, b) synchronous, c) regularly sampled.
- can be generated by **different processes**.

Notation:

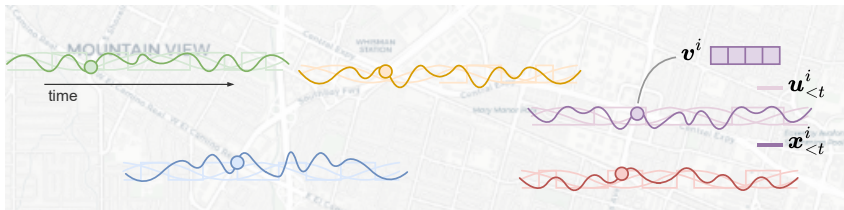
$$\mathcal{X}_t = \langle \mathbf{X}_t, \mathbf{U}_t, \mathbf{V} \rangle$$

$$\mathcal{X}_{<t} = [\mathcal{X}_0, \dots, \mathcal{X}_{t-2}, \mathcal{X}_{t-1}]$$

! Assumptions a),b),c) can be relaxed.

Example: Traffic monitoring system

Consider a sensor network monitoring the speed of vehicles at crossroads.



- $X_{<t}$ collects past traffic speed measurements.
- U_t stores identifiers for time-of-the-day and day-of-the-week.
- V collects static sensor's features, e.g., type or number of lanes of the monitored road.

→ Strong dependencies among time series that reflect the road network.

Forecasting models

Time series forecasting

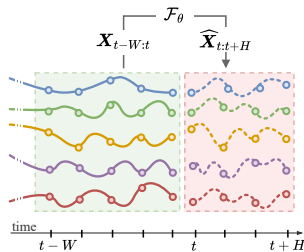
Multi-step time-series forecasting

Given a window of $W \geq 1$ **past** values

$$\mathcal{X}_{t-W:t} = [\mathcal{X}_{t-W}, \dots, \mathcal{X}_{t-1}],$$

predict $H \geq 1$ **future** observations

$$\mathbf{X}_{t+h} \quad h = 1, \dots, H.$$



In particular, we are interested in **learning a parametric model** $\mathcal{F}(\cdot; \theta)$ s.t.

$$\mathcal{F}(\mathcal{X}_{t-W:t}, \mathbf{U}_{t:t+H}; \theta) = \widehat{\mathbf{X}}_{t:t+H} \approx E_p[\mathbf{X}_{t:t+H}].$$

Probabilistic predictors can be considered as well, but we focus on point forecasts.

Training objective

For point predictors, parameters θ can be learned by minimizing a cost function $\ell(\cdot, \cdot)$ (e.g., MSE) on a training set

$$\begin{aligned}\hat{\theta} &= \arg \min_{\theta} \frac{1}{NT} \sum_{t=1}^T \ell \left(\widehat{\mathbf{X}}_{t:t+H}, \mathbf{X}_{t:t+H} \right) \\ &= \arg \min_{\theta} \frac{1}{NT} \sum_{t=1}^T \left\| \mathbf{X}_{t:t+H} - \widehat{\mathbf{X}}_{t:t+H} \right\|_2^2.\end{aligned}$$

Global and local predictors

Local models



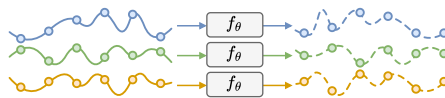
$$\hat{x}_{t+h}^i = f(x_{t-W:t}^i, \dots; \theta^i)$$

Example: Box-Jenkins method

😊 Tailored to each time series.

😞 Inefficient.

Global models



$$\hat{x}_{t+h}^i = f(x_{t-W:t}^i, \dots; \theta)$$

Example: DeepAR [1]

😊 Sample efficient.

😊 Allows for more complex models.

😞 Both approaches neglect dependencies among time series.

[1] Salinas *et al.*, “DeepAR: Probabilistic forecasting with autoregressive recurrent networks”, IJF 2020.

[4] Montero-Manso *et al.*, “Principles and algorithms for forecasting groups of time series: Locality and globality”, IJF 2021.

Accounting for spatial dependencies

- One option is to consider the input as **single multivariate time series**
 - Resulting predictors are **local**: $\widehat{\mathbf{X}}_{t+h} = f(\mathbf{X}_{t-W:t}, \dots; \boldsymbol{\theta})$.
 - ☹ Poor **sample complexity** and **scalability**.
- Models **operating on sets of time series** would allow to keep parameters shared.
 - Resulting predictors are **global**: $\widehat{\mathbf{X}}_{t+h}^{\mathcal{S}} = \mathcal{F}(\mathbf{X}_{t-W:t}^{\mathcal{S}}, \dots; \boldsymbol{\theta}), \quad \forall \mathcal{S} \subseteq \mathcal{D}$
 - 😊 Can be implemented by **attention-based** models (e.g, **Transformers**)
 - ☹ Does not exploit structural priors, high **computational** and **sample complexity**.
- Other methods (e.g., [5]) rely on **dimensionality reduction** to extract **shared latent factors**.
 - ☹ **Local** and **relational information** is **lost** and can still suffer from **scalability** issues.

[2] Benidis *et al.*, “Deep Learning for Time Series Forecasting: Tutorial and Literature Survey”, ACM CSUR 2022.

[5] Sen *et al.*, “Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting”, NeurIPS 2019.

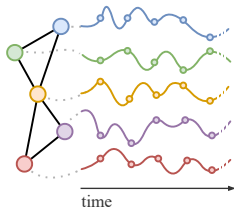
Graph-based forecasting

Relational information

💡 Exploit **functional dependencies** as an **inductive bias** to improve the forecasts.

We can model pairwise relationships existing at time step t with **adjacency matrix** $\mathbf{A}_t \in \{0, 1\}^{N \times N}$.

- $\mathbf{A}_t$ can be **asymmetric** and **dynamic** (can vary with t).



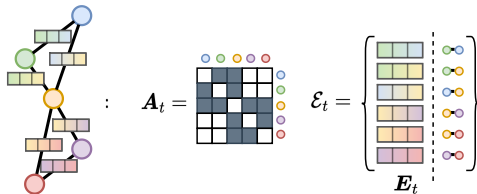
$$\mathbf{A}_t = \begin{matrix} & \begin{matrix} \text{blue} & \text{green} & \text{yellow} & \text{purple} & \text{red} \end{matrix} \\ \begin{matrix} \text{blue} \\ \text{green} \\ \text{yellow} \\ \text{purple} \\ \text{red} \end{matrix} & \begin{bmatrix} \text{white} & \text{dark blue} & \text{dark blue} & \text{white} & \text{white} \\ \text{dark blue} & \text{white} & \text{white} & \text{white} & \text{white} \\ \text{white} & \text{white} & \text{dark blue} & \text{white} & \text{white} \\ \text{white} & \text{white} & \text{white} & \text{dark blue} & \text{white} \\ \text{white} & \text{white} & \text{white} & \text{white} & \text{dark blue} \end{bmatrix} \end{matrix}$$

Relational information with attributes

Optional **edge attributes** $e_t^{ij} \in \mathbb{R}^{d_e}$ can be associated to each non-zero entry of A_t .

The **set of attributed edges** is denoted by

$$\mathcal{E}_t \doteq \{ \langle (i, j), e_t^{ij} \rangle \mid \forall i, j : A_t[i, j] \neq 0 \}.$$



→ Edge attributes can be both **categorical** or **numerical**.

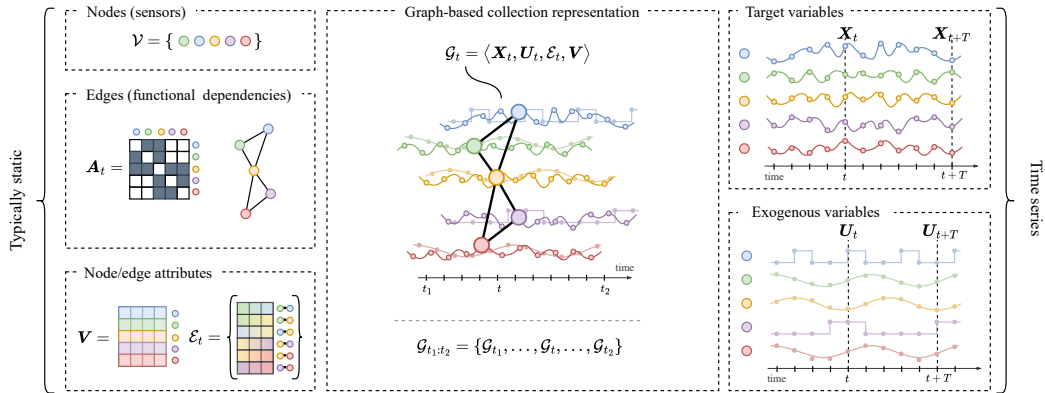
Example: Traffic monitoring system

Consider again the sensor network of the previous example.



- Edges in $\mathcal{E}$ can be obtained by considering the **road network**.
 - Road closures and traffic diversions can be accounted for with a dynamic topology $\mathcal{E}_t$.

Graph-based representations for correlated time series



$\mathcal{G}_t \doteq \langle X_t, U_t, \mathcal{E}_t, V \rangle$ contains the available information w.r.t. time step t .

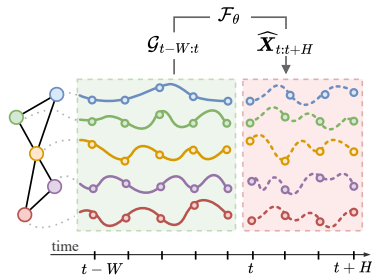
Relational inductive biases for time series forecasting

Forecasts can be conditioned on the available relational information $\mathcal{E}_{t-W:t}$

$$\widehat{\mathbf{X}}_{t:T+H}^S = \mathcal{F}(\mathcal{G}_{t-W:t}^S, \mathbf{U}_{t:t+H}^S; \theta) \quad \forall S \in \mathcal{D}$$

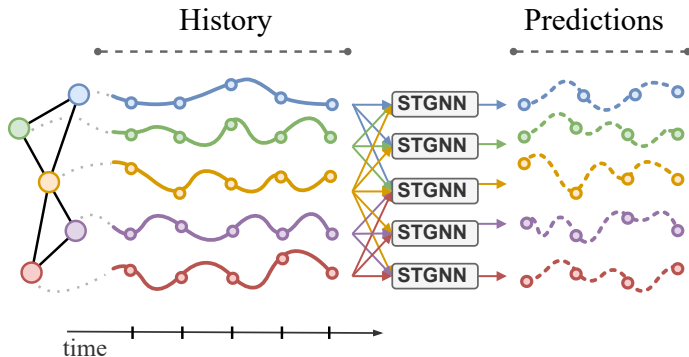
The conditioning can act as a **regularization** to localize predictions w.r.t. **each node**.

- 😊 Relational priors **prune spurious correlations**.
- 😊 More **scalable** than standard multivariate models.
- 😊 Can forecast any **subset** of correlated time series.



Spatiotemporal graph neural networks

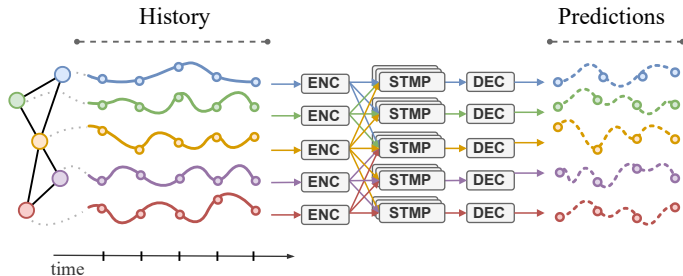
We call **spatiotemporal graph neural networks (STGNNs)** a neural network exploiting both temporal and spatial relations among input time series.



We focus on models based on message passing (MP).

A general recipe for building STGNNs

As a blueprint, we consider STGNNs consisting of three main components



- $\text{ENC}(\cdot)$ is the **encoding** layer, e.g., implemented by an MLP.
- $\text{STMP}(\cdot)$ is a stack of **spatiotemporal message-passing (STMP)** layers.
- $\text{DEC}(\cdot)$ is the **readout** layer, e.g., implemented by an MLP.

Spatiotemporal message passing

STMP blocks can be defined as:

$$h_t^{i,l+1} = \text{UP}^l \left(h_{\leq t}^{i,l}, \text{AGGR}_{j \in \mathcal{N}_t(i)} \left\{ \text{MSG}^l(h_{\leq t}^{i,l}, h_{\leq t}^{j,l}, e_{\leq t}^{ji}) \right\} \right)$$

Each block processes **sequences** while accounting for **relational dependencies**.

As in standard MP operators:

- $\text{MSG}^l(\cdot)$ is a **message function**, e.g., implemented by *temporal convolutional layers*.
- $\text{AGGR}\{\cdot\}$ is a permutation invariant **aggregation function**.
- $\text{UP}^l(\cdot)$ is an **update function**, e.g., implemented by an RNN.

! Blocks can be implemented by composing MP and sequence modeling operators.

→ Many possible designs exist.

[3] Cini et al., “Graph Deep Learning for Time Series Forecasting”, preprint 2023.

[6] Gilmer et al., “Neural message passing for quantum chemistry”, ICML 2017.

Design paradigms for STGNNs

Depending on the implementation of the STMP blocks, we categorize STGNNs into:

- **Time-and-space (T&S)**

Temporal and spatial processing cannot be factorized in two separate steps.

- **Time-then-space (TTS)**

Each time series is embedded in a vector and then representations are propagated on the graph.

- **Space-then-time (STT)**

Spatial propagation is performed before processing the resulting time series.

Time-and-space

In T&S models, representations at every node and time step are obtained by **jointly** propagating representation through time and space.

$$\mathbf{H}_{t-1}^{l+1} = \text{STMP}^l \left(\mathbf{H}_{\leq t-1}^l, \mathcal{E}_{\leq t-1} \right)$$

Several options exist.

- Integrate MP into neural operators for sequential data.
 - **Graph recurrent architectures, spatiotemporal convolutions, spatiotemporal attention, ...**
- Use sequence modelin operators to compute messages.
 - **Temporal graph convolutions, spatiotemporal cross-attention, ...**
- Product graph representations.

Example 1: From Recurrent Neural Networks...

Consider a **standard GRU cell** [7].

$$\mathbf{r}_t^i = \sigma \left(\Theta_r \left[\mathbf{x}_t^i || \mathbf{h}_{t-1}^i \right] + \mathbf{b}_r \right) \quad (1)$$

$$\mathbf{u}_t^i = \sigma \left(\Theta_u \left[\mathbf{x}_t^i || \mathbf{h}_{t-1}^i \right] + \mathbf{b}_u \right) \quad (2)$$

$$\mathbf{c}_t^i = \tanh \left(\Theta_c \left[\mathbf{x}_t^i || \mathbf{r}_t^i \odot \mathbf{h}_{t-1}^i \right] + \mathbf{b}_c \right) \quad (3)$$

$$\mathbf{h}_t^i = (1 - \mathbf{u}_t^i) \odot \mathbf{c}_t^i + \mathbf{u}_t^i \odot \mathbf{h}_{t-1}^i \quad (4)$$

Time series are processed **independently** for each node or as a **single multivariate** time series.

[7] Chung *et al.*, “Empirical evaluation of gated recurrent neural networks on sequence modeling” 2014.

...to Graph Convolutional Recurrent Neural Networks

We can obtain a T&S model by implementing the gates of the GRU with MP blocks:

$$\mathbf{Z}_t^l = \mathbf{H}_t^{l-1} \quad (5)$$

$$\mathbf{R}_t^l = \sigma \left(\text{MP}_r^l \left([\mathbf{Z}_t^l || \mathbf{H}_{t-1}^l], \mathcal{E}_t \right) \right), \quad (6)$$

$$\mathbf{O}_t^l = \sigma \left(\text{MP}_o^l \left([\mathbf{Z}_t^l || \mathbf{H}_{t-1}^l], \mathcal{E}_t \right) \right), \quad (7)$$

$$\mathbf{C}_t^l = \tanh \left(\text{MP}_c^l \left([\mathbf{Z}_t^l || \mathbf{R}_t^l \odot \mathbf{H}_{t-1}^l], \mathcal{E}_t \right) \right), \quad (8)$$

$$\mathbf{H}_t^l = \mathbf{O}_t^l \odot \mathbf{H}_{t-1}^l + (1 - \mathbf{O}_t^l) \odot \mathbf{C}_t^l, \quad (9)$$

These T&S models are known as **graph convolutional recurrent neural networks (GCRNNs)** [8].

[8] Seo *et al.*, “Structured sequence modeling with graph convolutional recurrent networks”, ICONIP 2018.

Popular GCRNNs

The **first GCRNN** has been introduced in [8], with **message passing (MP) blocks** implemented as polynomial graph convolutional filters.

GCRNNs have become popular in traffic forecasting with the **Diffusion Convolutional Recurrent Neural Network (DCRNN)** architecture [9].

DCRNN relies on a **bidirectional diffusion convolution**:

$$H'_t = \sum_{k=0}^K \left(D_{t,\text{out}}^{-1} A_t \right)^k H_t \Theta_1^{(k)} + \left(D_{t,\text{in}}^{-1} A_t^\top \right)^k H_t \Theta_2^{(k)} \quad (10)$$

[8] Seo *et al.*, “Structured sequence modeling with graph convolutional recurrent networks”, ICONIP 2018.

[9] Li *et al.*, “Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting”, ICLR 2018.

Example 2: Spatiotemporal convolutional networks (i)

Spatiotemporal convolutional networks (STCNs) instead **alternate spatial and temporal convolutions**:

1. Compute intermediate representations by using a **temporal convolutional** layer:

$$z_{t-W:t}^{i,l} = \text{TCN}^l \left(h_{t-W:t}^{i,l-1} \right) \quad \forall i$$

where TCN^l indicates a temporal convolutional layer.

2. Compute the updated representation at each time step by using a **graph convolution**:

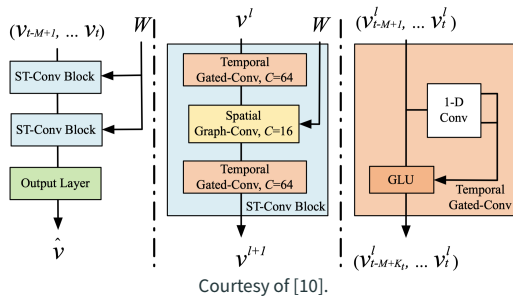
$$H_t^l = \text{MP}^l \left(Z_t^l, \mathcal{E}_t \right) \quad \forall t$$

Spatiotemporal convolutional networks (ii)

The first example of such architecture is the **STGCN** by Yu et al. [10].

The model is obtained by stacking STMP blocks consisting of

- a (gated) temporal convolution;
- a polynomial graph convolution;
- a second (gated) temporal convolution.



More advanced implementations exist, e.g., see **Graph Wavenet** [11].

[10] Yu et al., “Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting”, IJCAI 2018.

[11] Wu et al., “Graph wavenet for deep spatial-temporal graph modeling”, IJCAI 2019.

Example 3: Temporal Graph Convolution

A more integrated approach instead consists of implementing a **temporal propagation** mechanism in the message function.

For example, we can design STMP layers s.t.

$$\mathbf{h}_{t-W:t}^{i,l} = \text{TCN}_1^l \left(\mathbf{h}_{t-W:t}^{i,l-1}, \text{AGGR}_{j \in \mathcal{N}_t(i)} \left\{ \text{TCN}_2^l \left(\mathbf{h}_{t-W:t}^{i,l-1}, \mathbf{h}_{t-W:t}^{j,l-1}, \mathbf{e}_{t-W:t}^{ji} \right) \right\} \right).$$

💡 Analogous models can be built with **any sequence modeling architecture**.

→ **Example:** many rely on attention-based operators [12][13].

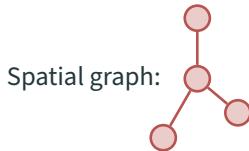
[12] Marisca *et al.*, “Learning to Reconstruct Missing Data from Spatiotemporal Graphs with Sparse Observations”, NeurIPS 2022.

[13] Wu *et al.*, “TraverseNet: Unifying Space and Time in Message Passing for Traffic Forecasting”, TNNLS 2022.

Example 4: Product graph representations

An alternative option is to consider the sequence $\mathcal{G}_{t-W:t}$ as a **single graph** with **temporal** and **spatial** edges.

In particular, **product graph representations** can be obtained by **combining the two edge sets**.



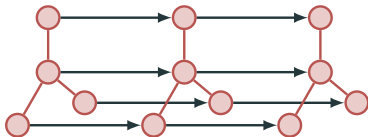
The resulting graph can be processed by any MP neural network.

[14] Sabbaqi *et al.*, “Graph-time convolutional neural networks: Architecture and theoretical analysis”, TPAMI 2023.

Building product graph representations

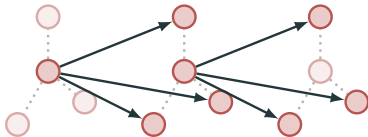
- **Cartesian product**

Spatial graphs are kept and each node is connected to itself in the previous time instant.



- **Kronecker product**

Each node is connected **only** to its neighbors in the previous time instant.

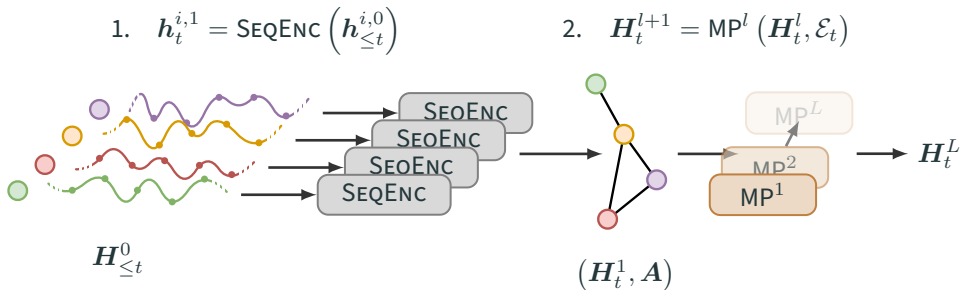


- ...

Time-then-Space models

The general recipe for a TTS model consists in:

1. **Embedding** each node-level time series in a vector.
2. **Propagating** obtained encodings throughout the graph with a stack of MP layers.



Pros & Cons of TTS models

Pros: 😊 Easy to implement and computationally efficient.

😊 We can reuse operators we already know.

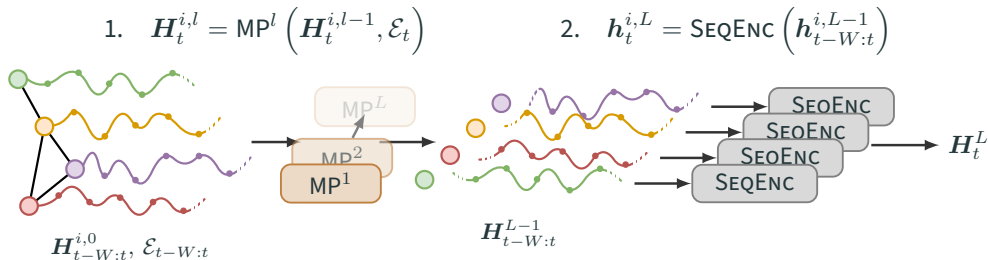
Cons: 😞 The 2-step encoding might introduce information bottlenecks.

😞 Accounting for changes in topology and dynamic edge attributes can be more problematic.

Space-then-Time

In STT approaches the two processing steps of TTS models are inverted:

1. Observations are **propagated among nodes** w.r.t. each time step using a stack of MP layers.
2. Each sequence of representations is processed by a **sequence encoder**.

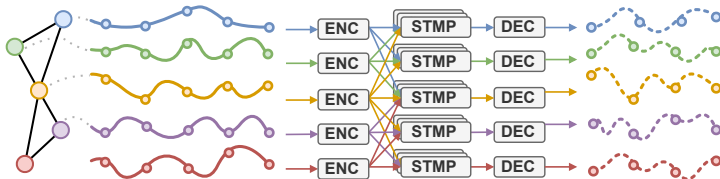


☹ They do not have the same computational advantages of TTS models.

Challenges

Ch. 1: Dealing with local dynamics

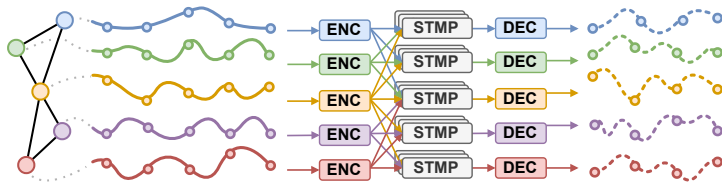
Globality and locality in STGNNs



Standard STGNNs are **global** models.

- 😊 Can handle arbitrary node sets.
 - 😊 Neighbors provide further conditioning on the predictions.
 - 😞 Might struggle with local effects.
 - 😞 Might need **long windows** and **high model capacity**.
- 💡 Use hybrid **global-local** STGNNs.

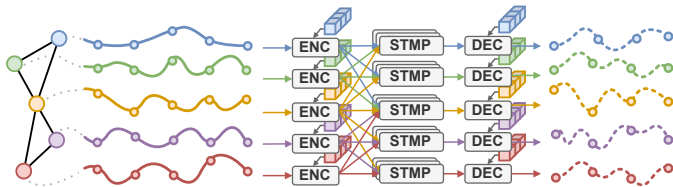
Global-local STGNNs



💡 We can turn some global components of the architecture into local.

- 😊 Resulting models can capture local effects.
- 😞 Might require a large number of local parameters.

Global-local STGNNs with node embeddings

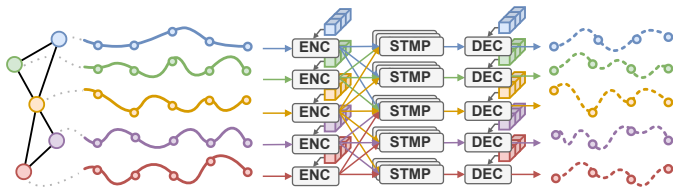


Node embeddings can amortize the learning of local components.

Node embeddings are a table of **learnable parameters** $Q \in \mathbb{R}^{N \times d_q}$ associated with **each node**.

- 😊 Fed into encoder/decoder, **amortize** the learning of **local components**.
- 😊 Most of the model's **parameters remain shared**.
- 😊 Number of parameters scales **linearly** with the number of time series . . .
 - One might consider **intermediate solutions**, e.g., learning embeddings for **clusters** of time series.

Transferability



! Hybrid global-local STGNNs are not inductive models.

However, the cost of transfer learning can be reduced.

- 😊 Keep shared parameters fixed and finetune local parameters only.
- 😊 Node embeddings can be regularized to facilitate transfer further.

Empirical results on benchmark datasets

MODELS	MetrLA	PemsBAY	CER-E	AQI	MetrLA	PemsBAY	CER-E	AQI
Reference arch.	Global models				+ Local node embeddings			
RNN	3.54 $\pm$.00	1.77 $\pm$.00	4.57 $\pm$.01	14.02 $\pm$.04	3.15 $\pm$.03	1.59 $\pm$.00	4.22 $\pm$.02	13.73 $\pm$.04
GCRNN-IMP	3.35 $\pm$.01	1.70 $\pm$.01	4.44 $\pm$.01	12.87 $\pm$.02	3.10 $\pm$.01	1.59 $\pm$.00	4.18 $\pm$.01	12.48 $\pm$.03
RNN+IMP	3.34 $\pm$.01	1.72 $\pm$.00	4.39 $\pm$.01	12.74 $\pm$.02	3.08 $\pm$.01	1.58 $\pm$.00	4.12 $\pm$.03	12.33 $\pm$.02
GCRNN-AMP	3.22 $\pm$.02	1.65 $\pm$.00	4.57 $\pm$.04	12.29 $\pm$.02	3.07 $\pm$.02	1.59 $\pm$.00	4.17 $\pm$.02	12.17 $\pm$.05
RNN+AMP	3.24 $\pm$.01	1.66 $\pm$.00	4.31 $\pm$.01	12.30 $\pm$.02	3.06 $\pm$.01	1.58 $\pm$.01	4.13 $\pm$.01	12.15 $\pm$.02
Baseline arch.	Original				+ Local node embeddings			
DCRNN	3.22 $\pm$.01	1.64 $\pm$.00	4.28 $\pm$.01	12.96 $\pm$.03	3.07 $\pm$.02	1.60 $\pm$.00	4.13 $\pm$.02	12.53 $\pm$.02
GraphWaveNet	3.05 $\pm$.03	1.56 $\pm$.01	3.97 $\pm$.01	12.08 $\pm$.11	2.99 $\pm$.02	1.58 $\pm$.00	4.01 $\pm$.01	11.81 $\pm$.04
AGCRN	3.16 $\pm$.01	1.61 $\pm$.00	4.45 $\pm$.01	13.33 $\pm$.02	3.14 $\pm$.00	1.62 $\pm$.00	4.37 $\pm$.02	13.28 $\pm$.03

Table 1: MAE on benchmark datasets.

Transfer learning results

We consider datasets coming from **four different traffic networks**.

→ **One of the networks is left out** at training time and used for evaluating **transferability**.

RNN+IMP		PEMS03	PEMS04	PEMS07	PEMS08
Fine-tuning	Global	15.30 $\pm$ 0.03	21.59 $\pm$ 0.11	23.82 $\pm$ 0.03	15.90 $\pm$ 0.07
	Embeddings	14.64 $\pm$ 0.05	20.27 $\pm$ 0.11	22.23 $\pm$ 0.08	15.45 $\pm$ 0.06
	– Variational	14.56 $\pm$ 0.03	20.19 $\pm$ 0.05	22.43 $\pm$ 0.02	15.41 $\pm$ 0.06
	– Clustering	14.60 $\pm$ 0.02	19.91 $\pm$ 0.11	22.16 $\pm$ 0.07	15.41 $\pm$ 0.06
Zero-shot		18.20 $\pm$ 0.09	23.88 $\pm$ 0.08	32.76 $\pm$ 0.69	20.41 $\pm$ 0.07

Table 2: Transfer learning results (MAE) after fine-tuning on a week of data.

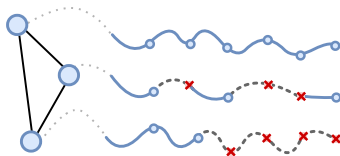
Ch. 2: Missing data

The problem of missing data

Time series collected in the real world often are **incomplete** and have **missing data**.

Most time series analysis tools operate on **complete sequences**.

- ⚠️ We need a way to **impute**, i.e., *reconstruct*, missing data.
- 😊 We can **exploit the available relational structure**.

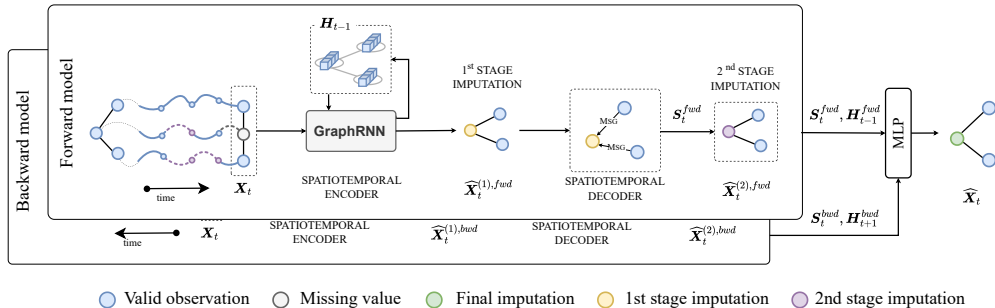


[16] **Cini** et al., “Filling the G_ap_s: Multivariate Time Series Imputation by Graph Neural Networks”, ICLR 2022.

[12] Marisca, **Cini**, et al., “Learning to Reconstruct Missing Data from Spatiotemporal Graphs with Sparse Observations”, NeurIPS 2022.

[17] De Felice, **Cini**, et al., “Graph-based Virtual Sensing from Sparse and Partial Multivariate Observations”, ICLR 2024.

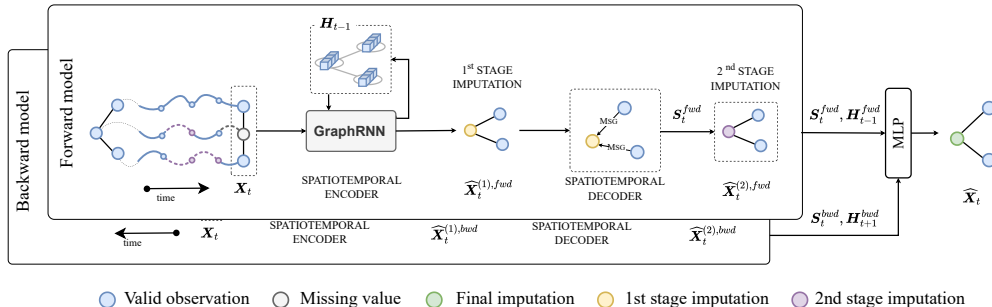
Graph Recurrent Imputation Network (GRIN)



GRIN [16] is a bidirectional autoregressive STGNN.

- A **graph RNN** encodes the sequence of **valid observations**.
- An **final message-passing decoder** accounts for concurrent observations at neighbors

Limitations

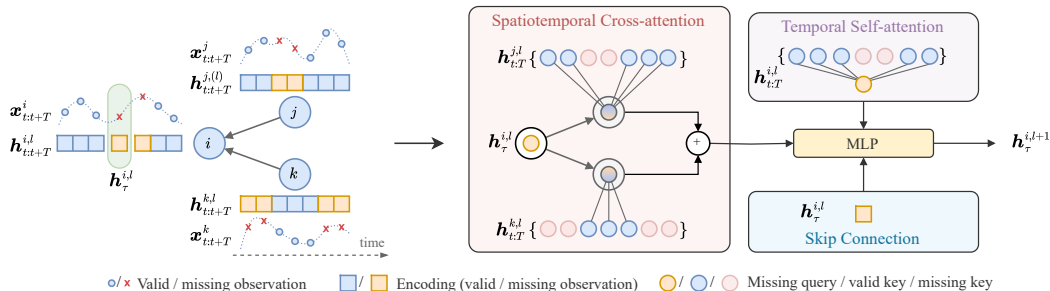


😊 Extremely effective in many settings.

😞 However, autoregressive models can **accumulate reconstruction error**.

[16] Cini et al., "Filling the Gaps: Multivariate Time Series Imputation by Graph Neural Networks", ICLR 2022.

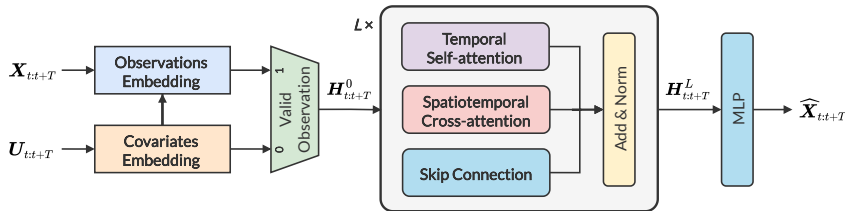
Spatiotemporal Point Inference Network (SPIN)



💡 Use **attention-based operators**.

- 😊 We can exploit **cross-attention** to propagate information **among related time series**.
- 😊 Prevents **autoregressive error accumulation**.

Spatiotemporal Point Inference Network (SPIN)



😊 Effective even in scenarios with highly sparse observations.

😞 Can be computationally expensive.

→ hierarchical computation can help by reducing the cost from $\mathcal{O}(T^2|\mathcal{E}|)$ to $\mathcal{O}(TK|\mathcal{E}|)$ 😊

Some empirical results

We compared the introduced **graph-based** methods against several state-of-the-art approaches including standard **attention-based** architectures and popular **RNN** models.

MODEL	Block missing		Point missing		Simulated failures	
	PEMS-BAY	METR-LA	PEMS-BAY	METR-LA	AQI-36	AQI
Mean	5.46 $\pm$.00	7.48 $\pm$.00	5.42 $\pm$.00	7.56 $\pm$.00	53.48 $\pm$.00	39.60 $\pm$.00
VAR	2.09 $\pm$.10	3.11 $\pm$.08	1.30 $\pm$.00	2.69 $\pm$.00	15.64 $\pm$.08	22.95 $\pm$.30
rGAIN	2.18 $\pm$.01	2.90 $\pm$.01	1.88 $\pm$.02	2.83 $\pm$.01	15.37 $\pm$.26	21.78 $\pm$.50
BRITS	1.70 $\pm$.01	2.34 $\pm$.01	1.47 $\pm$.00	2.34 $\pm$.00	14.50 $\pm$.35	20.21 $\pm$.22
SAITS	1.56 $\pm$.01	2.30 $\pm$.01	1.40 $\pm$.03	2.26 $\pm$.00	18.16 $\pm$.42	21.33 $\pm$.15
STTr	1.70 $\pm$.02	3.54 $\pm$.00	0.74 $\pm$.00	2.16 $\pm$.00	11.98 $\pm$.53	18.11 $\pm$.25
GRIN	1.14 $\pm$.01	2.03 $\pm$.00	0.67$\pm$.00	1.91$\pm$.00	12.08 $\pm$.47	14.73 $\pm$.15
SPIN	1.06$\pm$.02	1.98$\pm$.01	0.70 $\pm$.01	1.90$\pm$.01	11.77 $\pm$.54	13.92$\pm$.15
SPIN-H	1.05$\pm$.01	2.05 $\pm$.02	0.73 $\pm$.01	1.96 $\pm$.03	10.89$\pm$.27	14.41 $\pm$.13

Table 3: Reconstruction MAE.

Sensitivity analysis

MODEL	METR-LA			PEMS-BAY			AQI		
	Missing rate			Missing rate			Missing rate		
	50 %	75 %	95 %	50 %	75 %	95 %	50 %	75 %	95 %
BRITS	2.52 $\pm$.00	3.02 $\pm$.00	5.19 $\pm$.02	1.55 $\pm$.00	2.17 $\pm$.00	3.91 $\pm$.02	14.90 $\pm$.03	18.29 $\pm$.03	29.83 $\pm$.07
SAITS	2.48 $\pm$.00	3.74 $\pm$.01	6.72 $\pm$.01	1.50 $\pm$.00	2.96 $\pm$.01	7.40 $\pm$.01	15.36 $\pm$.02	20.64 $\pm$.05	34.57 $\pm$.05
STTr	2.31 $\pm$.00	2.71 $\pm$.00	5.13 $\pm$.01	0.85 $\pm$.00	1.13 $\pm$.00	2.70 $\pm$.01	9.11 $\pm$.02	12.56 $\pm$.05	25.65 $\pm$.11
GRIN	2.05 $\pm$.00	2.39 $\pm$.00	4.08 $\pm$.02	0.79 $\pm$.00	1.09 $\pm$.00	2.70 $\pm$.01	8.43 $\pm$.01	10.97 $\pm$.02	20.38 $\pm$.10
SPIN	2.02 $\pm$.00	2.24 $\pm$.00	2.89 $\pm$.01	0.79 $\pm$.00	1.00 $\pm$.00	1.71 $\pm$.00	8.15 $\pm$.01	9.96 $\pm$.02	15.51 $\pm$.08
SPIN-H	2.01 $\pm$.00	2.20 $\pm$.00	2.82 $\pm$.00	0.79 $\pm$.00	0.97 $\pm$.00	1.68 $\pm$.00	8.67 $\pm$.02	10.27 $\pm$.02	15.75 $\pm$.07

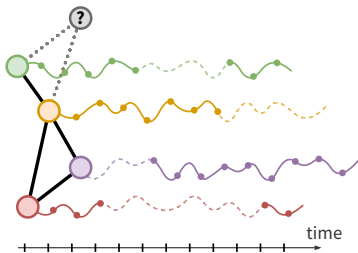
Table 4: MAE with increasing data sparsity in the *Point missing* settings.

Virtual sensing

Virtual sensing consists in estimating values at unmonitored locations

Graph-based models are particularly effective:

- 😊 We can condition estimates on neighboring sensors.
- 😊 Global models can handle unseen nodes and edges.
- 😊 Very useful when sensing is expensive.



[18] Wu *et al.*, “Inductive Graph Neural Networks for Spatiotemporal Kriging”, AAAI 2021.

[17] De Felice, **Cini**, *et al.*, “Graph-based Virtual Sensing from Sparse and Partial Multivariate Observations”, ICLR 2024.

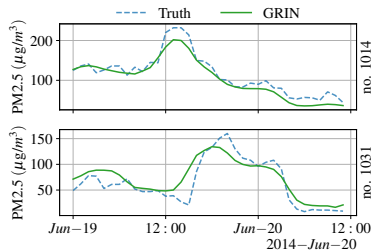
Graph-based virtual sensing

💡 Add a **virtual node** with **no data** and let the model **infer** the corresponding time series.

Clearly, several assumptions are needed

- high degree of homogeneity of sensors,
- capability to reconstruct from observations at neighboring sensors,
- and many more...

Two virtual sensors for air quality. (from [16])



[16] **Cini et al.**, “Filling the G_ap_s: Multivariate Time Series Imputation by Graph Neural Networks”, ICLR 2022.

[12] Marisca, **Cini**, et al., “Learning to Reconstruct Missing Data from Spatiotemporal Graphs with Sparse Observations”, NeurIPS 2022.

Ch. 3: Scalability

Computational scalability

Correlated time series must be processed along **two dimensions**:

- Space** – dimension corresponding to the **number of time series** (sensors);
- & **Time** – dimension corresponding to the **number of time steps** (number of acquired observations).

Computational complexity of STGNNs

W : length of time series · N : n. of nodes · $|\mathcal{E}_t|$: n. of edges · L : n. of MP layers

The computational complexity of T&S models is given by:

- node-wise temporal processing – $\mathcal{O}(WN)$; $\rightarrow \mathcal{O}(W(N + L|\mathcal{E}_t|))$
- L MP layers **for each time step** – $\mathcal{O}(WL|\mathcal{E}_t|)$.

TTS are more scalable:

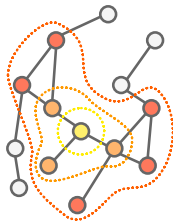
- node-wise temporal processing – $\mathcal{O}(WN)$; $\rightarrow \mathcal{O}(WN + L|\mathcal{E}_t|)$
- L MP layers **at the last time step** – $\mathcal{O}(L|\mathcal{E}_t|)$.

There are **two main approaches** to mitigating this issue.

Graph subsampling

Computations can be reduced by training on **subgraphs** of the full network.

- sampling the **K -hop neighborhood** of a subset of nodes;
- **rewiring** the graph to reduce the total number of edges.



Mostly adapted from methods developed in **static graph processing**.

- ☹ Subsampling might break long-range spatiotemporal dependencies.
- ☹ The learning signal may be noisy.

[19] Gandhi *et al.*, “Spatio-Temporal Multi-graph Networks for Demand Forecasting in Online Marketplaces” 2021.

[20] Hamilton *et al.*, “Inductive representation learning on large graphs”, NeurIPS 2017.

[21] Rong *et al.*, “DropEdge: Towards Deep Graph Convolutional Networks on Node Classification”, ICLR 2020.

Pre-computation

Pre-processing methods (e.g., [22]) enable scalability to large graphs by:

- precomputing a representation for each node's neighborhood ahead of training;
- processing the obtained node representations as if they were **i.i.d. samples**.

An extension to spatiotemporal data is given by SGP [23], which acts in 2 steps:

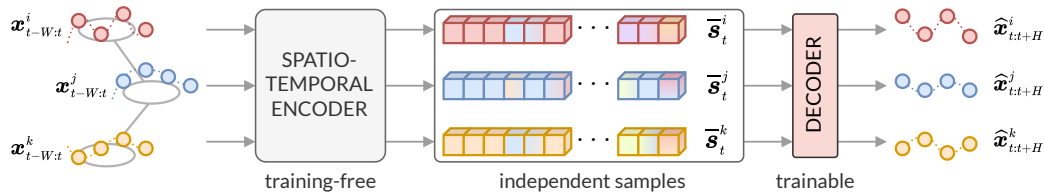
1. obtain a temporal encoding at each time step with a deep echo state network¹;
2. propagate such encodings through the graph using powers of a graph shift operator.

[22] Frasca *et al.*, “SIGN: Scalable inception graph neural networks” 2020.

[23] Cini *et al.*, “Scalable Spatiotemporal Graph Neural Networks”, AAAI 2023.

¹ A randomized recurrent neural networks

SGP: Scalable Graph Predictor



SGP is based on an **encoder-decoder** architecture.

- ☺ The **decoder** is the **only trainable component** (can be a standard **MLP**).
- ☺ Any **propagation** through time and space happens at the **encoding** step.
- ☺ Encoded observations can then be **sampled uniformly**.
- ☺ The **cost of a training step** becomes **independent of W and $|\mathcal{E}_t|$** .

Empirical results

MODELS	METR-LA					PEMS-BAY				
	15 m	30 m	60 m	Average		15 m	30 m	60 m	Average	
	MAE	MAE	MAE	MAE	MAPE	MAE	MAE	MAE	MAE	MAPE
LSTM	2.99 $\pm$.00	3.58 $\pm$.00	4.43 $\pm$.01	3.58 $\pm$.00	1.19 $\pm$.05	1.39 $\pm$.00	1.83 $\pm$.01	2.35 $\pm$.01	1.79 $\pm$.00	4.16 $\pm$.05
FC-LSTM	3.33 $\pm$.01	3.43 $\pm$.01	3.67 $\pm$.01	3.46 $\pm$.01	1.15 $\pm$.09	2.22 $\pm$.01	2.25 $\pm$.01	2.34 $\pm$.02	2.26 $\pm$.01	5.33 $\pm$.04
DynGESN	3.27 $\pm$.00	3.99 $\pm$.00	5.00 $\pm$.00	3.98 $\pm$.00	11.11 $\pm$.01	1.57 $\pm$.00	2.13 $\pm$.01	2.81 $\pm$.02	2.09 $\pm$.01	4.74 $\pm$.01
DCRNN	2.82 $\pm$.00	3.23 $\pm$.01	3.74 $\pm$.01	3.20 $\pm$.00	8.88 $\pm$.05	1.36 $\pm$.00	1.71 $\pm$.00	2.08 $\pm$.01	1.66 $\pm$.00	3.76 $\pm$.01
GWNet	2.72 $\pm$.01	3.10 $\pm$.02	3.54 $\pm$.03	3.06 $\pm$.02	8.40 $\pm$.03	1.31 $\pm$.00	1.64 $\pm$.01	1.94 $\pm$.01	1.58 $\pm$.00	3.58 $\pm$.02
FC-GGN	2.72 $\pm$.01	3.05 $\pm$.01	3.44 $\pm$.01	3.01 $\pm$.00	8.27 $\pm$.00	1.32 $\pm$.00	1.63 $\pm$.01	1.89 $\pm$.01	1.56 $\pm$.01	3.51 $\pm$.03
UG-GGN	2.72 $\pm$.00	3.10 $\pm$.00	3.54 $\pm$.01	3.06 $\pm$.00	8.40 $\pm$.04	1.33 $\pm$.00	1.67 $\pm$.01	1.99 $\pm$.01	1.61 $\pm$.01	3.59 $\pm$.03
SGP	2.69 $\pm$.00	3.05 $\pm$.00	3.45 $\pm$.00	3.00 $\pm$.00	8.27 $\pm$.02	1.30 $\pm$.00	1.60 $\pm$.00	1.88 $\pm$.00	1.54 $\pm$.00	3.44 $\pm$.01

Table 5: Results on benchmark traffic datasets.

😊 SGP matches the **state of the art** in popular benchmarks . . .

Empirical results

MODELS		Prediction error (MAE)			Resource utilization		
		30 mins	7 hours 30 mins	11 hours	Batch/s	Memory	Batch size
PV-US 100	DCRNN	1.39 ± 0.09	3.34 ± 0.22	3.54 ± 0.48	2.04 ± 0.01	9.63 GB	2
	GWNet	1.45 ± 0.13	5.09 ± 0.63	5.26 ± 1.34	2.01 ± 0.02	11.64 GB	2
	GatedGN	1.33 ± 0.08	2.94 ± 0.05	3.12 ± 0.14	8.41 ± 0.09	11.46 GB	5
	SGP	1.09 ± 0.01	3.14 ± 0.21	3.16 ± 0.19	116.58 ± 8.74	2.21 GB	4096
PV-US Full	DCRNN	1.59 ± 0.17	4.10 ± 0.27	4.93 ± 0.60	1.37 ± 0.00	11.59 GB	1*
	GWNet	1.65 ± 0.23	6.93 ± 0.58	7.93 ± 0.17	0.77 ± 0.00	11.35 GB	2
	GatedGN	1.61 ± 0.06	3.25 ± 0.04	3.04 ± 0.05	8.83 ± 0.10	11.14 GB	1*
	SGP	1.09 ± 0.00	3.06 ± 0.11	3.13 ± 0.13	118.64 ± 8.35	2.21 GB	4096

Table 6: Results on PV-US (5,016 nodes and from 420K (100-NN) to 3.7M (Full) edges).

... while having vastly **lower memory consumption** and **higher throughput** 😊

- ☹️ Considerable amount of **disk storage required** for the pre-computed representations.
- ☹️ More **reliant on hyperparameter selection** than end-to-end approaches.

Conclusions

Some Takeaways

Deep Learning
for **time series**

+

Deep Learning
on **graphs**

- ⚡ **Relational** inductive **biases** allow for exploiting dependencies among the time series,
- 😊 ... and deal with **irregularities in time and space**.
- 😊 Whenever possible, **global-local models** are a safe starting point.
- 💡 More challenges discussed in the **full tutorial**: graph learning, model quality assessment, ...

Challenges. Local effects • Missing data • Scalability

tsl: Torch Spatiotemporal



tsl (Torch Spatiotemporal) is a python library built upon **PyTorch** and **PyG** to accelerate research on neural spatiotemporal data processing methods, with a focus on **Graph Neural Networks**.

 torch-spatiotemporal.readthedocs.io

 github.com/TorchSpatiotemporal/tsl



Notebook

Spatiotemporal Graph Neural Networks with tsl



Contributors



Andrea Cini



Ivan Marisca



Daniele Zambon



Cesare Alippi



Graph Deep Learning for Time Series Forecasting

Andrea Cini¹, Ivan Marisca¹, Daniele Zambon¹, and Cesare Alippi^{1,2}

¹The Swiss AI Lab IDSIA USI-SUPSI & Università della Svizzera italiana, Switzerland.

²Politecnico di Milano, Italy.

{andrea.cini, ivan.marisca, daniele.zambon, cesare.alippi}@usi.ch

Abstract

Graph-based deep learning methods have become popular tools to process collections of correlated time series. Differently from traditional multivariate forecasting methods, neural graph-based predictors take advantage of pairwise relationships by conditioning forecasts on a (possibly dynamic) graph spanning the time series collection. The conditioning can take the form of an architectural inductive bias on the neural forecasting architecture, resulting in a family of deep learning models called spatiotemporal graph neural networks. Such relational inductive biases enable the training of global forecasting models on large time-series collections, while at the same time localizing predictions w.r.t. each element in the set (i.e., graph nodes) by accounting for local correlations

THE END

Questions?

References i

- [1] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, **“DeepAR: Probabilistic forecasting with autoregressive recurrent networks,”** *International Journal of Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.
- [2] K. Benidis, S. S. Rangapuram, V. Flunkert, *et al.*, **“Deep learning for time series forecasting: Tutorial and literature survey,”** *ACM Comput. Surv.*, vol. 55, no. 6, Dec. 2022, ISSN: 0360-0300. doi: 10.1145/3533382. [Online]. Available: <https://doi.org/10.1145/3533382>.
- [3] **A. Cini**, I. Marisca, D. Zambon, and C. Alippi, **“Graph deep learning for time series forecasting,”** *arXiv preprint arXiv:2310.15978*, 2023.
- [4] P. Montero-Manso and R. J. Hyndman, **“Principles and algorithms for forecasting groups of time series: Locality and globality,”** *International Journal of Forecasting*, vol. 37, no. 4, pp. 1632–1653, 2021.
- [5] R. Sen, H.-F. Yu, and I. S. Dhillon, **“Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting,”** *Advances in Neural Information Processing Systems*, vol. 32, 2019.

References ii

- [6] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “**Neural message passing for quantum chemistry,**” in *International conference on machine learning*, PMLR, 2017, pp. 1263–1272.
- [7] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “**Empirical evaluation of gated recurrent neural networks on sequence modeling,**” *arXiv preprint arXiv:1412.3555*, 2014.
- [8] Y. Seo, M. Defferrard, P. Vandergheynst, and X. Bresson, “**Structured sequence modeling with graph convolutional recurrent networks,**” in *International Conference on Neural Information Processing*, Springer, 2018, pp. 362–373.
- [9] Y. Li, R. Yu, C. Shahabi, and Y. Liu, “**Diffusion convolutional recurrent neural network: Data-driven traffic forecasting,**” in *International Conference on Learning Representations*, 2018.
- [10] B. Yu, H. Yin, and Z. Zhu, “**Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting,**” in *Proceedings of the 27th International Joint Conference on Artificial Intelligence*, 2018, pp. 3634–3640.

References iii

- [11] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, **“Graph wavenet for deep spatial-temporal graph modeling,”** in *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 2019, pp. 1907–1913.
- [12] I. Marisca, **A. Cini**, and C. Alippi, **“Learning to reconstruct missing data from spatiotemporal graphs with sparse observations,”** in *Advances in Neural Information Processing Systems*, 2022.
- [13] Z. Wu, D. Zheng, S. Pan, Q. Gan, G. Long, and G. Karypis, **“Traversenet: Unifying space and time in message passing for traffic forecasting,”** *IEEE Transactions on Neural Networks and Learning Systems*, 2022.
- [14] M. Sabbaqi and E. Isufi, **“Graph-time convolutional neural networks: Architecture and theoretical analysis,”** *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 12, pp. 14 625–14 638, Dec. 2023, ISSN: 1939-3539. DOI: 10.1109/TPAMI.2023.3311912.
- [15] **A. Cini**, I. Marisca, D. Zambon, and C. Alippi, **“Taming local effects in graph-based spatiotemporal forecasting,”** *arXiv preprint arXiv:2302.04071*, 2023.

References iv

- [16] **A. Cini**, I. Marisca, and C. Alippi, “**Filling the g_ap_s: Multivariate time series imputation by graph neural networks,**” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=k0u3-S3wJ7>.
- [17] G. De Felice, **A. Cini**, D. Zambon, V. Gusev, and C. Alippi, “**Graph-based Virtual Sensing from Sparse and Partial Multivariate Observations,**” in *International Conference on Learning Representations*, 2024.
- [18] Y. Wu, D. Zhuang, A. Labbe, and L. Sun, “**Inductive Graph Neural Networks for Spatiotemporal Kriging,**” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 4478–4485.
- [19] A. Gandhi, Aakanksha, S. Kaveri, and V. Chaoji, “**Spatio-temporal multi-graph networks for demand forecasting in online marketplaces,**” in *Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track: European Conference, ECML PKDD 2021, Bilbao, Spain, September 13–17, 2021, Proceedings, Part IV*, 2021, pp. 187–203, ISBN: 978-3-030-86513-9. DOI: 10.1007/978-3-030-86514-6_12. [Online]. Available: https://doi.org/10.1007/978-3-030-86514-6_12.

References v

- [20] W. Hamilton, Z. Ying, and J. Leskovec, **“Inductive representation learning on large graphs,”** *Advances in neural information processing systems*, vol. 30, 2017.
- [21] Y. Rong, W. Huang, T. Xu, and J. Huang, **“Dropedge: Towards deep graph convolutional networks on node classification,”** in *International Conference on Learning Representations*, 2020.
- [22] F. Frasca, E. Rossi, D. Eynard, B. Chamberlain, M. Bronstein, and F. Monti, **“SIGN: Scalable inception graph neural networks,”** *arXiv preprint arXiv:2004.11198*, 2020.
- [23] **A. Cini**, I. Marisca, F. M. Bianchi, and C. Alippi, **“Scalable spatiotemporal graph neural networks,”** *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 6, pp. 7218–7226, Jun. 2023. doi: 10.1609/aaai.v37i6.25880.
- [24] A. Kazi, L. Cosmo, S.-A. Ahmadi, N. Navab, and M. M. Bronstein, **“Differentiable graph module (DGM) for graph convolutional networks,”** *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 2, pp. 1606–1617, 2022.

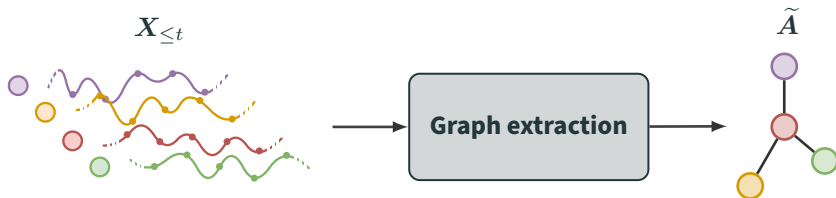
References vi

- [25] **A. Cini**, D. Zambon, and C. Alippi, “**Sparse graph learning from spatiotemporal time series,**” *Journal of Machine Learning Research*, vol. 24, no. 242, pp. 1–36, 2023.
- [26] T. Kipf, E. Fetaya, K.-C. Wang, M. Welling, and R. Zemel, “**Neural relational inference for interacting systems,**” in *International conference on machine learning*, PMLR, 2018, pp. 2688–2697.

Bonus

Learning an adjacency matrix

- ☹ Relational information is **not** always **available**.
- 😊 Relational architectural **biases** can nonetheless be exploited
→ **extract a graph** from the time series or node attributes

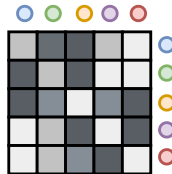


- When possible, the learned graph should be **sparse**.
- It can be interpreted as **regularizing** a **spatial attention** operator.

Time-series similarities

Probably, the simplest approach to extract a graph from the time series is by computing **time series similarity scores**.

- Pearson correlation
- Correntropy
- Granger causality
- Kernels for time series
- ...



→ Thresholding might be necessary to obtain binary and sparse graphs.

End-to-end latent graph learning

An integrated approach: **learn the graph end-to-end** with the downstream task

→ e.g., by minimizing the forecasting error (MAE, MSE...).

Two different formulations:

1. learning directly an **adjacency matrix** $\mathbf{A} \in \mathbb{R}^{N \times N}$;
2. learning a **probability distribution over graphs** p_{Φ} generating $\mathbf{A}$ (often $\in \{0, 1\}^{N \times N}$).



One key challenge is keeping both $\mathbf{A}$ and the subsequent computations **sparse**.

→ **non-trivial** with gradient-based optimization.

Direct approach

A direct approach consists in learning $\tilde{\mathbf{A}}$ as function $\xi(\cdot)$ of edge scores $\Phi \in \mathbb{R}^{N \times N}$ as

$$\tilde{\mathbf{A}} = \xi(\Phi)$$

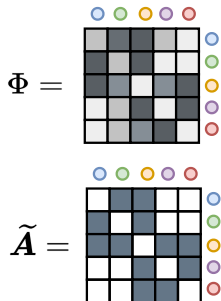
Edge scores Φ

- can be a table of **learnable** model **parameters**,
- obtained as a **function** of the **inputs** and/or other parameters:

$$\Phi = \Phi(\mathbf{X}, \tilde{\Phi}).$$

Function $\xi(\cdot)$ can enforce structures on $\tilde{\mathbf{A}}$, like,

→ make $\tilde{\mathbf{A}}$ **binary**, a **k-NN** graph, a **tree**...



Edge score factorization

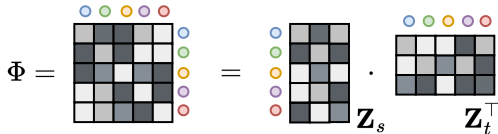
The number of possible edge scores is **quadratic** in the number of nodes ($\Phi \in \mathbb{R}^{N \times N}$)

→ a common approach is to factorize Φ :

$$\tilde{A} = \xi(\Phi) \quad \Phi = \mathbf{Z}_s \mathbf{Z}_t^\top$$

with

- $\mathbf{Z}_s \in \mathbb{R}^{N \times d}$ **source** node embeddings
- $\mathbf{Z}_t \in \mathbb{R}^{N \times d}$ **target** node embeddings


$$\Phi = \mathbf{Z}_s \cdot \mathbf{Z}_t^\top$$

$\mathbf{Z}_s$ and $\mathbf{Z}_t$ can be learned as tables of (local) parameters or **as a function of the input window**.

[11] Wu *et al.*, “Graph wavenet for deep spatial-temporal graph modeling”, IJCAI 2019.

Pro & Cons of the direct approach

- 😊 Easy to **implement**.
- 😊 Many possible **parametrizations**.
- 😊 Edge scores are usually **easy to learn** end-to-end.
- 😞 It often results in **dense computations** with $\mathcal{O}(N^2)$ complexity.
- 😞 Sparsifying A results in **sparse gradients**.
- 😞 Encoding prior structural information requires **smart parametrizations**.

Probabilistic methods

In this context, probabilistic methods aim at learning a **parametric distribution** p_{Φ} for $\mathbf{A}$.

- Different parametrizations of p_{Φ} allow for embedding **graph structural priors** on the sampled graphs, e.g., edge density, bound node degrees.

Independent edges

For every edge (i, j)

$$\mathbf{A}_{i,j} \sim \text{Bernoulli}(\sigma(\Phi_{i,j})).$$

Fixed-degree graphs

For each node i , sample w/o replacement k nodes from

$$\text{Categorical}(\text{SoftMax}(\Phi_{i,1}, \dots, \Phi_{i,N})).$$

- As seen before, Φ can be factorized and p_{Φ} made input dependent, e.g.,

$$\Phi = \xi(\mathbf{Z}_s \mathbf{Z}_t^{\top}), \quad \mathbf{A} \sim p_{\Phi}(\mathbf{A} | \mathbf{X}_{<t}, \mathbf{U}_{<t}, \mathbf{V}).$$

[24] Kazi et al., “Differentiable graph module (DGM) for graph convolutional networks”, TPAMI 2022.

[25] Cini et al., “Sparse graph learning from spatiotemporal time series”, JMLR 2023.

Learning graph distributions

Training losses average over all graphs according to p_Φ , e.g., based on point predictions

$$\mathcal{L}(\Phi) = \mathbb{E}_{\mathbf{A} \sim p_\Phi} \left[\ell \left(\widehat{\mathbf{X}}_{t:t+H}, \mathbf{X}_{t:t+H} \right) \right], \quad \mathcal{L}(\Phi) = \ell \left(\mathbb{E}_{\mathbf{A} \sim p_\Phi} \left[\widehat{\mathbf{X}}_{t:t+H} \right], \mathbf{X}_{t:t+H} \right),$$

where $\mathbf{X}_{t:t+H} = \mathcal{F}(\mathbf{X}_{t-W:t}, \mathbf{A}; \theta)$.

More generally, comparing predictive distributions by means of a divergence measure Δ

$$\mathcal{L}(\Phi) = \Delta \left(p_\Phi(\widehat{\mathbf{X}}_{t:t+H}), p(\mathbf{X}_{t:t+H}) \right).$$



Gradient-based optimization requires computing $\nabla_\Phi \mathcal{L}(\Phi)$,
→ i.e., differentiating w.r.t. the parameters of the integrated distribution.



Analytical computations is often unfeasible;



Monte Carlo approximations require care.

Monte Carlo gradient estimators

💡 One approach is to **reparametrize** $\tilde{A} \sim p_{\Phi}(A)$ as: $\tilde{A} = g(\Phi, \varepsilon)$, $\varepsilon \sim p(\varepsilon)$
decoupling parameters Φ from the random component ε : $\nabla_{\Phi} \mathcal{L}(\Phi) = \mathbb{E}_{\varepsilon} \left[\nabla_{\Phi} \ell(\widehat{\mathbf{X}}, \mathbf{X}) \right]$.

😊 Practical and **easy** to implement,

😞 rely on **continuous relaxations** and make subsequent computations scale with $\mathcal{O}(N^2)$.

💡 Conversely, **score-function** (SF) gradient estimators rely on the relation

$$\nabla_{\Phi} \mathbb{E}_{p_{\Phi}} \left[\ell(\widehat{\mathbf{X}}, \mathbf{X}) \right] = \mathbb{E}_{p_{\Phi}} \left[\ell(\widehat{\mathbf{X}}, \mathbf{X}) \nabla_{\Phi} \log p_{\Phi} \right]$$

😞 suffer from **high variance** (use variance reduction techniques),

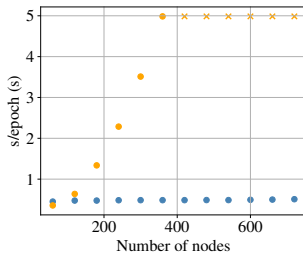
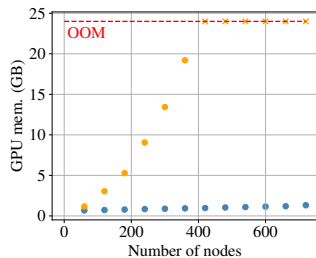
😊 allow to **keep computations sparse** through the model.

→ we can use **Monte Carlo** estimator.

[26] Kipf *et al.*, “Neural relational inference for interacting systems”, ICML 2018.

[25] Cini *et al.*, “Sparse graph learning from spatiotemporal time series”, JMLR 2023.

Computational efficiency



- Score-function
- Pathwise

With score-based gradient estimators $\nabla_{\Phi} \mathcal{L}(\Phi) = \mathbb{E}_{\mathbf{A} \sim p_{\Phi}} \left[\ell(\widehat{\mathbf{X}}, \mathbf{X}) \nabla_{\Phi} \log p_{\Phi}(\mathbf{A}) \right]$.

- 😊 They are **computationally efficient** as ∇_{Φ} is computed with respect to $\log p_{\Phi}(\mathbf{A})$.
 - do not rely on continuous relaxation of **discrete random variables**;
 - allow for **sparse message passing** to compute $\widehat{\mathbf{X}}$ (and, in turn, of $\ell(\widehat{\mathbf{X}}, \mathbf{X})$) by rely on sparse matrices $\mathbf{A}$.
- 😞 They can be sample inefficient due to the **high variance** of the gradient estimates.